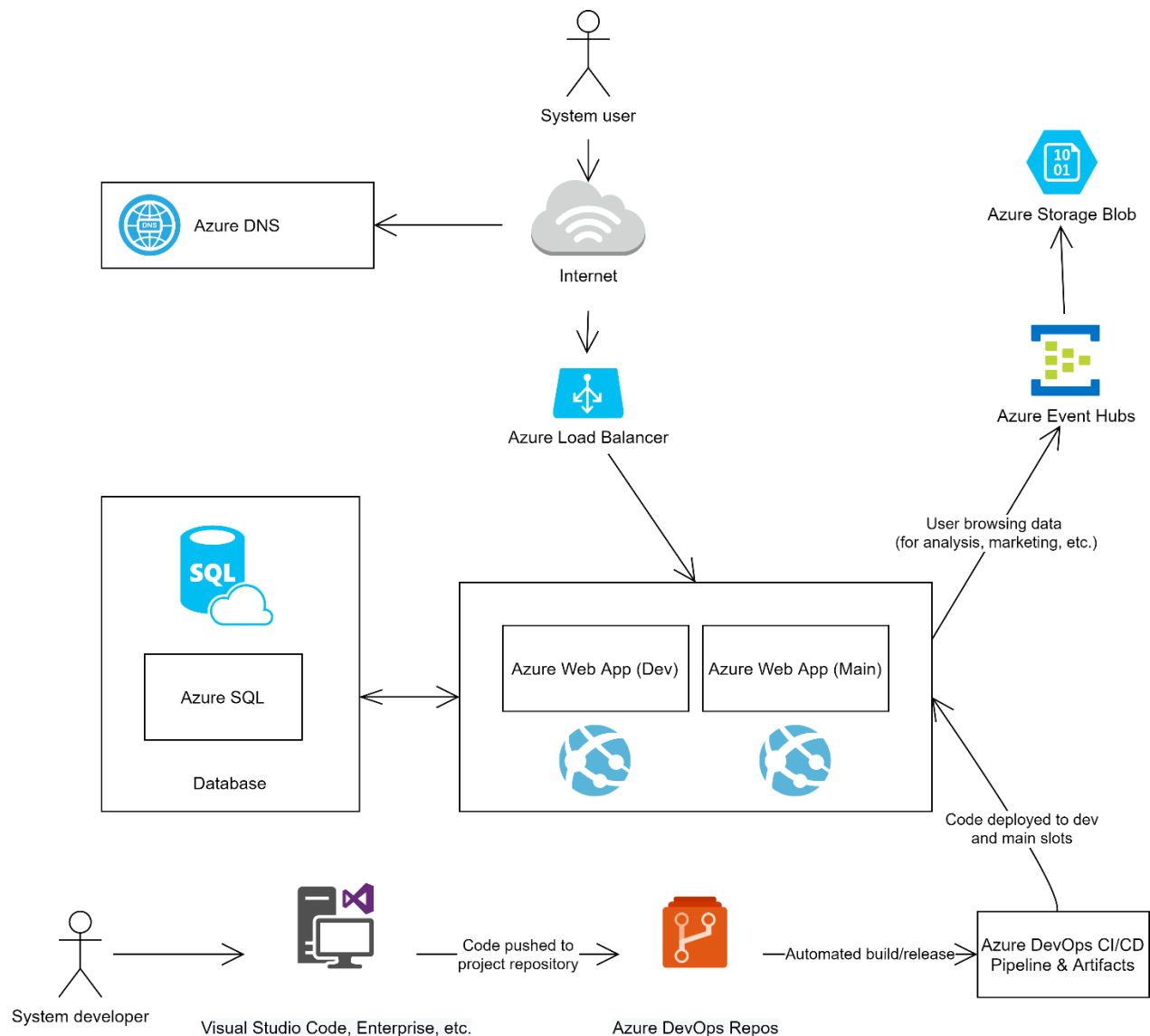


The designed system is a simple web application, i.e., a landing page for another upcoming project.

It consists of a one-pager website that informs user of the features of the upcoming project, project team, contact information, etc. The purpose of the landing page is to inform people of a soon-to-be-released project, its release date, feature list, advantages and so on, and to attract as many potential new users as possible, via encouragements to subscribe to newsletters, share on social media, etc. Apart from its informative purpose, it has some interactive functionality, i.e., entering email for joining the mailing list and subscribing to a newsletter. Furthermore, the user browsing data, such as the amount of time users spend visiting the page, the amount of newsletter subscribers, the website heatmap (where users click and spend most time viewing page components) are collected for analysis and marketing purposes. A simple single database would be used for two-way data flows – to store user email information (for the mailing list participants that input their email information on the website) and some information that is displayed on the website dynamically, e.g., number of subscribers, some blog post data, etc. A separate storage would be used for one-way data flow, i.e., for user browsing data.



The proposed architecture would make use of Microsoft Azure services.

From the development side, an assumption is made that a single web developer is working on the project (website developed using Node.js) using an IDE (Visual Studio Code, Enterprise, etc.) and pushes or pulls code that is on Azure DevOps repository. Let's assume that the website has one repository with two branches – dev and main, for testing and reviewing minor changes, and for the main version of the page respectively. An Azure DevOps CI/CD Pipeline with Artifacts is setup for getting the source from repository, installing tools and packages (if any), building the project, running tests (if any), and packaging/publishing packages (such as npm) via Artifacts. The website is deployed to Dev and Main Azure Web Apps, based on dev and main branches respectively.

From the system user side, they may access the website via the Internet using a computer, a tablet or smartphone. The website may be entered by entering a domain provided by Azure DNS service. As there are two Web Apps (one for development purposes, and the main one), two DNS may be setup, as example: dev.somewebsite.com and somewebsite.com respectively. A large influx of users may be expected, therefore to maintain the availability of the website and smooth performance, an Azure Load Balancer is used to distribute load evenly. Azure SQL service may be used for the database, and as the designed system is rather simple, a single database is enough, with at least two tables (one for storing user email information for sending newsletters and one for other miscellaneous information dynamically displayed on the website). The data flow is two-way, as the website may not only write information to the database, but also pull information from it (info such as stats, blog posts, etc.). Azure Event Hubs and Storage Blob services are used for storing system user browsing tendencies, for example, data of buttons clicked, the amount of time spent on the website, the amount of successful newsletter subscriptions, etc. Azure Event hubs in particular is used to process such events, which may be programmed either by JavaScript using the installed Event Hub npm package, or by integrating a different solution, such as Google Analytics/Google Tag Manager. Lastly, the event data is captured to Azure Blob Storage. Data is in Apache Avro format, and it can be viewed in Azure Storage Explorer (not displayed in the designed system diagram as it is a separate downloadable tool and there are alternative ways to work with the info).